Table of Contents

Deployment guide — CPL Project Feasibility Calculator on Azure

Audience: CPL IT / cloud engineers. **Version:** 1.0 (November 2026). **What this is:** the steps to install the Project Feasibility Calculator in CPL's own Azure tenant, using the files in this kit. A first-time deployment takes roughly half a day, most of it waiting on app-registration approvals and DNS.

> **Read this first — what has and hasn't been tested.**

Part	Status
Application code, database migrations, seed scripts	Fully tested (203 automated tests; 6 browser tests; fresh-database install rehearsed).
Production build and the container's runtime layout	Built and served successfully. **The Dockerfile itself has not been built** (no Docker engine was available to the builder). Expect to run it once and fix anything small.
Bicep templates (`infra/main.bicep`)	Compile and lint cleanly. **Never deployed** to a live subscription. Review names, SKUs and policies with your cloud team before running.
Sign-in with Microsoft Entra ID	Coded to Auth.js's documented behaviour; **never exercised against a real tenant.**
Sending email through Microsoft Graph	Unit-tested against Microsoft's documented request shapes; **never sent for real.** The "Test email" step below is the first real check.

> Section 11 lists what to verify at go-live. Please treat any failure there as expected first-run friction, not a sign the application is broken, and tell the builder.

---

1. What you are installing

A Next.js (Node 22) web application with a PostgreSQL database. Staff sign in with their CPL Microsoft account. Users score and route fragrance project briefs; approvers sign off the requirements each brief raises; Admins manage people, roles and scoring rules.

```
Staff browser ──HTTPS──▶ App Service (Linux container)  ──▶ Azure
Database for PostgreSQL
                                  │   (VNet integrated)
(private, no public access)
                                  ├─▶ Key Vault (secrets, via managed
identity)
```

```
                                  ├─▶ Microsoft Graph (sends notification
email)
                                  └─▶ Application Insights / Log Analytics
Logic App (every 5 min) ──GET /api/notifications/dispatch──▶ App
Service
```

Azure resource	Created by `infra/main.bicep`	Notes
Virtual network, 3 subnets	yes	app, database, one-off jobs
PostgreSQL Flexible Server 16	yes	private access only, 14-day backups
Container Registry (Basic)	yes	holds the app image
App Service plan + Web App (Linux, container)	yes	health check on `/api/health`
Key Vault (RBAC, purge protection)	yes	5 secrets
User-assigned managed identity	yes	pulls image, reads Key Vault
Log Analytics + Application Insights	yes	90-day retention
Logic App (Consumption)	yes	the scheduled job that sends queued emails
Two Entra app registrations	**no — you create them**	sign-in, and mail
Shared mailbox for outgoing mail	**no — you create it**	e.g. `feasibility@cplaromas.com`
DNS name and certificate	**no**	optional custom domain

If CPL already has standard patterns (hub-spoke networking, a shared ACR, Private Endpoints, Front Door/WAF), substitute them: the application only needs the environment variables in section 12.

2. What is in the kit

```
feasibility-handover/
  Dockerfile                  # 'runtime' (the app) and 'tools'
(migrate/seed jobs) targets
  infra/main.bicep            # Azure resources
  infra/main.bicepparam       # parameter file to fill in
  scripts/migrate.mjs         # applies database migrations
  drizzle/                    # the SQL migrations (0000 – 0004)
  src/                        # application source (includes seed
data)
  docs/deployment-guide.md    # this document
  docs/user-guide/            # user guides for every role
  docs/runbook.md, architecture.md, backlog.md, rules-for-simon.md
```

3. Prerequisites

- An Azure subscription and a resource group; you hold **Owner** (or Contributor + User Access Administrator) on it, because the template creates role assignments.
- Rights to create **app registrations** in the CPL Entra tenant and grant **admin consent**.
- Rights to create a **shared mailbox** and run Exchange Online PowerShell (for the mail restriction in step 3).
- A workstation with: Azure CLI (2.60+), `az bicep`, Docker (or use `az acr build`), and `openssl`. Optional: Node 22 if you prefer to run migrations without Docker.
- A decision on the app's URL. The default is `https://<app-name>.azurewebsites.net`; a custom domain such as `feasibility.cplaromas.com` is nicer. The sign-in app registration's redirect URI must match whichever you choose.

4. Step 1 — Choose names and generate secrets

```
export RG=rg-cpl-feasibility
export LOCATION=uksouth
export PREFIX=cplfeas            # 3-12 lowercase letters/digits

export AUTH_SECRET=$(openssl rand -hex 32)
export CRON_SECRET=$(openssl rand -hex 32)
export POSTGRES_ADMIN_PASSWORD=$(openssl rand -base64 24 | tr -d
'/+=')
```

Keep these in your password manager; they are also stored in Key Vault by the template.

5. Step 2 — Create the sign-in app registration (Entra ID)

1. Entra admin centre → **App registrations** → **New registration**. Name: `CPL Feasibility Calculator`. Supported account types: **Single tenant**.
2. **Redirect URI** (type *Web*): `https://<your-url>/api/auth/callback/microsoft-entra-id` (If you are using the default URL you don't know the exact app name until the template runs. You can add the real URI after step 5; the template prints it as the output `entraRedirectUri`.)
3. **Certificates & secrets** → **New client secret.** Copy the value. Put it in `ENTRA_CLIENT_SECRET`.
4. Note the **Application (client) ID** and **Directory (tenant) ID**.
5. Under **API permissions**, the default `User.Read` is enough. No other permissions are needed for sign-in.

Access control lives **inside the application** (the *People and roles* screen), not in Entra app roles. Anyone in your tenant who can sign in but has no role in the app sees a "your role doesn't submit briefs" message and an empty Briefs list. If you want to restrict sign-in to named people, set **Assignment required = Yes** on the enterprise application and assign the users.

6. Step 3 — Create the mail app registration and mailbox (Microsoft Graph)

The app sends notification emails (approval requests, "your brief is approved", veto and revoke notices) as a single shared mailbox.

1. Create a **shared mailbox**, e.g. `feasibility@cplaromas.com`.
2. Create a second **app registration**, `CPL Feasibility Mail`. Single tenant, no redirect URI. Create a client secret → `GRAPH_CLIENT_SECRET`.
3. **API permissions** → **Add** → **Microsoft Graph** → **Application permissions** → **`Mail.Send`**, then **Grant admin consent**.
4. **Restrict the app to that one mailbox** (otherwise `Mail.Send` allows sending as anyone). In Exchange Online PowerShell:

   ```
   New-DistributionGroup -Name "Feasibility Mail Senders" -Type
   Security -Members feasibility@cplaromas.com
   New-ApplicationAccessPolicy -AppId <mail-app-client-id> `
     -PolicyScopeGroupId "Feasibility Mail Senders" -AccessRight
   RestrictAccess `
     -Description "Feasibility Calculator may only send as
   feasibility@"
   Test-ApplicationAccessPolicy -AppId <mail-app-client-id> -
   Identity feasibility@cplaromas.com
   ```

   (Microsoft's current guidance may prefer RBAC for Applications in Exchange Online; either method is acceptable if it limits the app to this mailbox. Policies can take up to an hour to take effect.)

7. Step 4 — Deploy the infrastructure

Edit `infra/main.bicepparam`: set `namePrefix`, `entraTenantId`, `entraClientId`, `graphClientId`, `graphSender`, and `appServiceSku` (`B1` for a test environment, `P0v3` or larger for production). Secrets come from your environment variables.

```
az group create -n $RG -l $LOCATION

export ENTRA_CLIENT_SECRET='<from step 2>'
export GRAPH_CLIENT_SECRET='<from step 3>'

az deployment group create -g $RG \
  -f infra/main.bicep -p infra/main.bicepparam \
  --query properties.outputs
```

Save the outputs: `webAppName`, `webAppUrl`, `acrName`, `acrLoginServer`, `keyVaultName`, `postgresServer`, `jobsSubnetId`, `managedIdentityId`, `entraRedirectUri`.

If you used the default URL, go back to the sign-in app registration and add `entraRedirectUri` as a redirect URI now.

The web app will show an error until the image exists (next step). That is expected.

8. Step 5 — Build and publish the container image

```
ACR=<acrName from the outputs>

# Build in Azure (no local Docker needed):
az acr build -r $ACR -t feasibility:1.0.0 --target runtime .
az acr build -r $ACR -t feasibility-tools:1.0.0 --target tools .
```

Then point the app at it. Either re-run the deployment with `-p imageTag=1.0.0`, or:

```
az webapp config container set -g $RG -n <webAppName> \
  --container-image-name $ACR.azurecr.io/feasibility:1.0.0
az webapp restart -g $RG -n <webAppName>
```

(`feasibility-tools` is used once for the database in step 6 and is **not** a web app.)

9. Step 6 — Prepare the database (migrate and seed)

The database has **no public access**, so the one-off jobs must run from inside the virtual network. The `tools` image does the job; run it as an Azure Container Instance in the `snet-jobs` subnet the template created.

Get the connection string from Key Vault (you need *Secrets Officer/User* on it — the template grants Officer to `keyVaultAdminObjectId` if you set it):

```
DBURL=$(az keyvault secret show --vault-name <keyVaultName> -n
database-url --query value -o tsv)
```

Run the three jobs, in this order, each as its own container instance. The example uses the registry's admin-less pull via the managed identity:

```
run_job () {   # $1 = name, $2... = command
  NAME=$1; shift
  az container create -g $RG -n $NAME \
    --image $ACR.azurecr.io/feasibility-tools:1.0.0 \
    --assign-identity <managedIdentityId> --acr-identity
<managedIdentityId> \
    --subnet <jobsSubnetId> --restart-policy Never \
    --secure-environment-variables DATABASE_URL="$DBURL" \
    --command-line "$*"
  az container logs -g $RG -n $NAME
  az container delete -g $RG -n $NAME -y
}

run_job feas-migrate  node scripts/migrate.mjs
run_job feas-seed     npm run seed:apply
run_job feas-people   npm run seed:people
```

What each does:

Job	Effect	Safe to re-run?
`node scripts/migrate.mjs`	Creates/updates all tables (migrations 0000–0003).	Yes.
`npm run seed:apply`	Loads the roles and the scoring rule set **v1** (generated from the workbook).	Yes (skips if present).
`npm run seed:people`	Creates CPL's named people and their roles from `src/db/seed/people.ts`.	Yes (never removes roles).

Expected harmless output: Postgres `NOTICE` lines ("schema drizzle already exists", "identifier will be truncated").

> **If `az container create` options differ** on your CLI version, or CPL's policy forbids ACI, run the same three commands from any machine that can reach the database (a jump host or VPN-connected workstation): clone/unzip the kit, `npm ci`, set `DATABASE_URL`, and run `node scripts/migrate.mjs`, `npm run seed:apply`, `npm run seed:people`.

Check:

```
curl https://<your-url>/api/health          # {"status":"ok"}
curl "https://<your-url>/api/health?db=1"   #
{"status":"ok","database":"up"}
```

10. Step 7 — First sign-in and go-live set-up

1. Browse to the URL. The sign-in page shows a single **Sign in with Microsoft** button. Sign in as one of the seeded **Admins** (Alfred Muthunathan, Simon Roper or Tia O'Neil). The first sign-in links the person's Microsoft account to the entry created in step 6 (matching on their work email, ignoring case).

2. Open **People and roles** (Admin menu). Confirm every person and role is as CPL expects. Pauline Holmes holds the **General Manager (veto)** role.

3. **Tier T rule set.** The seeded rule set v1 has the legacy "A/T" tier. To split out Tier T (Target account — always needs Commercial Director approval), create the draft from a job, then publish it in the app:

   ```
   run_job feas-tier-t  npx tsx src/db/seed/add-tier-t-draft.ts
   alfred.muthunathan@cplaromas.com
   ```

   Then in the app: **Rule sets → the new draft → Run replay → Publish**. (With a new database there is no history, so the replay shows no changes.) The Tier T weight is assumed to be 100 (copied from Tier A); **confirm with CPL before publishing.**

4. **Add the people who submit briefs.** The seeded list contains only Admins and approvers. Admins and approvers **cannot submit briefs**; only the *Account Manager* and *Sales Coordinator* roles can. In *People and roles* add each sales colleague by work email and give them *Account Manager* (or *Sales Coordinator*; the form does not yet offer "submit on behalf of", so the two roles currently behave the same in the screens). CPL

has not yet supplied this list. For the go-live test, an Admin can temporarily give themselves *Account Manager*.

5. **Test email.** The app sends these emails: *Sign-off needed* (to the holders of each approval role when a brief is submitted), *Brief approved* and *Sign-off rejected* (to the submitter), *Pre-approval declared* and *revoked*, and *Brief vetoed* and *Veto reversed*. To test the mail path before go-live, set `EMAIL_ALLOWED_RECIPIENTS` (section 12) to your own address so nobody else is emailed, then as an Account Manager submit a brief with a short deadline (14 days or fewer) using a clearly labelled reference such as `TEST-GOLIVE`. Within about 5 minutes the Development Director's *"Sign-off needed — TEST-GOLIVE"* should be sent (or, with the allowlist, recorded as `suppressed` for anyone but you). **Remove `EMAIL_ALLOWED_RECIPIENTS` at go-live**; while it is set, mail to everyone else is silently held back.

6. Ask one approver and one Account Manager to sign in and walk the user guide.

11. Go-live checklist

- ☐ `/api/health?db=1` returns `ok`.
- ☐ An Admin can sign in with their real Microsoft account (not a test user). The page shows only the **Sign in with Microsoft** button.
- ☐ An account **not** in the people list can sign in but sees nothing and cannot open `/admin/*`.
- ☐ **`ALLOW_DEV_LOGIN` is not set** anywhere (Application settings, slot settings, Key Vault). The sign-in page must not show a "Test user" dropdown.
- ☐ The sales colleagues who will submit briefs are added with the Account Manager / Sales Coordinator role.
- ☐ A test brief submits and the score and routing look right.
- ☐ The Logic App's run history shows a green run every 5 minutes (HTTP 200).
- ☐ A queued email arrives from the shared mailbox; the Microsoft Graph restriction is in force (try sending as another mailbox: it must fail).
- ☐ Application Insights shows requests; no repeating exceptions.
- ☐ Database backup retention and a restore test are agreed (section 14).
- ☐ An Admin has rotated or confirmed every secret in Key Vault.
- ☐ Test data created during go-live checks is understood (briefs cannot be deleted; they are an audit record). Prefer doing step 10.4 with an obviously labelled customer reference such as `TEST-GOLIVE`.

12. Configuration reference

All set by the template as App Service settings (secrets are Key Vault references).

Setting	Purpose
`DATABASE_URL`	PostgreSQL connection string (`sslmode=require`).
`AUTH_SECRET`	Signs session tokens. Random, 32+ bytes.
`NEXTAUTH_URL`	The public URL, with no trailing slash. Also the base of links in

Setting	Purpose
	emails.
`AUTH_TRUST_HOST`	`true` (the app sits behind Azure's front end).
`AUTH_MICROSOFT_ENTRA_ID_ID` / `_SECRET` / `_ISSUER`	Sign-in. Issuer is `https://login.microsoftonline.com/<tenant>/v2.0`.
`EMAIL_PROVIDER`	`graph` for Azure. (`resend` is the test-site provider and is not used here.)
`GRAPH_TENANT_ID` / `GRAPH_CLIENT_ID` / `GRAPH_CLIENT_SECRET` / `GRAPH_SENDER`	Mail via Microsoft Graph.
`CRON_SECRET`	Shared secret the Logic App presents to `/api/notifications/dispatch`.
`EMAIL_ALLOWED_RECIPIENTS`	Optional safety net for test phases: comma-separated addresses or `@domain` entries. Mail to anyone else is marked `suppressed` and never sent. **Leave unset in production.**
`DB_POOL_MAX`	Max database connections per instance (default 5). With *n* instances the server sees up to *n* × 5.
`WEBSITES_PORT`	`3000`.
`ALLOW_DEV_LOGIN`	**Must be absent.** Enables a password-less test login.

13. Day-two operations

Deploying a new version. Build a new tag (`az acr build ... -t feasibility:1.0.1`), run `migrate` if the release notes say the schema changed, then `az webapp config container set ... --container-image-name ...:1.0.1`. Migrations are additive, so the previous version keeps working against a migrated database. Run `migrate` **before** switching the image, and keep the previous tag so you can switch back.

Logs. Application Insights → *Failures* / *Logs*; or Log Analytics tables `AppServiceConsoleLogs`, `AppServiceHTTPLogs`.

Scaling. The app is stateless (sessions are signed cookies). You can scale out the plan; remember `DB_POOL_MAX × instances` against the database connection limit.

Rotating secrets. Update the secret in Key Vault; then restart the web app (Key Vault references refresh on restart). For `cron-secret` also update the Logic App's `cronSecret` parameter (redeploy the template with the new value). Rotating `AUTH_SECRET` signs everyone out, which is harmless.

Adding or removing people. Admins do this in the app (*People and roles*). No IT action is needed. Remember that a role change takes effect when that person next signs in.

Backups and restore. Flexible Server keeps 14 days of point-in-time backups. Practise a restore to a new server once before go-live. The audit trail is part of the database; there is no separate store.

Support boundary. CPL IT owns Azure, identity and mail. Anything about scoring rules, who approves what, or role assignments is an Admin task in the app, not an IT task.

14. Moving existing data from the test site (Supabase)

The test deployment on Vercel/Supabase holds only test data. **Normally do not migrate it**: start Azure with a clean database (steps 6–7). If CPL wants to keep briefs created during testing:

```
pg_dump --no-owner --no-privileges --data-only --column-inserts \
  --exclude-table-data='drizzle.*' "$SUPABASE_DIRECT_URL" > data.sql
```

Run migrations on Azure first (step 6), then load `data.sql` from inside the network, **before** `seed:apply` (otherwise rule set v1 and roles are duplicated), and run `seed:people` afterwards. Test users (`*@cpl.example`) should then be deactivated in *People and roles*. This path has not been rehearsed; restore into a scratch server first.

15. Troubleshooting

Symptom	Likely cause and fix
Site shows "Application Error" or 503	Image not pulled or crashed. Web App → *Log stream*. Check the `acrPull` role assignment and that the tag exists in ACR.
Sign-in page says "No sign-in method is configured"	The three `AUTH_MICROSOFT_ENTRA_ID_*` settings are missing, or a Key Vault reference isn't resolving (Web App → *Configuration* shows a red/yellow icon). Check the identity has *Key Vault Secrets User*.

Symptom	Likely cause and fix
Microsoft sign-in loops back, or "redirect_uri mismatch" (AADSTS50011)	The redirect URI in the app registration must equal `<NEXTAUTH_URL>/api/auth/callback/microsoft-entra-id` exactly.
Signed in but "empty" app / nothing visible	The person has no role. An Admin adds one in *People and roles*; they sign out and in again.
"Sign-in refused" after Microsoft login	The account is deactivated in *People and roles*, or the work email was previously bound to a different Microsoft identity (e.g. a re-created account). An Admin must deactivate the old entry and add a fresh one.
Emails aren't arriving	Open the Logic App's run history: a `401` means the `CRON_SECRET` differs between the Logic App and the web app; a `200` whose output lists `failures` shows the error text for each failed email (e.g. Graph `ErrorAccessDenied`, which usually means the Application Access Policy hasn't propagated or `GRAPH_SENDER` is wrong). Rows are in the `notifications` table (`delivery_status` = `queued`, `sent`, `failed` or `suppressed`; `suppressed` means `EMAIL_ALLOWED_RECIPIENTS` held it back). **A `failed` row is not retried automatically**: after fixing the cause, set it back with `UPDATE notifications SET delivery_status='queued' WHERE delivery_status='failed';` and the next 5-minute run resends.
Migration	The container instance isn't in `snet-jobs`, or the `database-url` secret host differs from the server's FQDN.

Symptom	Likely cause and fix
job can't connect	
`too many clients` errors	Lower `DB_POOL_MAX`, or raise the server's connection limit or SKU.

16. Known limitations and what CPL should decide

1. **Roles are fixed at sign-in.** A role change applies the next time the person signs in (it does not cut off an already-open session). Deactivating a person blocks the next sign-in only.
2. **No line-manager data.** If an approver submits a brief they would be asked to approve, the requirement is escalated to an Admin (see the Admin guide).
3. **Notifications:** a failed email stays failed until someone re-queues it (see Troubleshooting); there is no automatic retry/back-off or status screen yet (on the backlog).
4. **No reminders.** Approvers are emailed once when a sign-off is raised. Nothing chases a sign-off that sits unactioned (on the backlog), and a person who becomes the holder of a role later is not emailed about sign-offs already open.
5. **Tier T weight** (assumed 100) and **Pauline Holmes's scope** (veto only; no other approval role) are CPL decisions still open.
6. **Not built:** automatic reminders for stale approvals; a role-assignment approval workflow; an export of the audit trail (the data is in the database).

17. Contacts

Builder / product owner: Alfred Muthunathan (CIO). Business rules: Simon Roper and Tia O'Neil (Admins). Rule definitions and their reasoning: `docs/rules-for-simon.md`.